Exact Scheduling of Parallel Additive Manufacturing Machines for Total Tardiness Minimization

***Abstract*— This paper studies the one-dimensional batch scheduling problem for parallel additive manufacturing (AM) machines with the objective of minimizing total tardiness. We develop a tailored exact branch-and-bound algorithm that branches on the assignment of parts to machines and evaluates each complete assignment by decomposing it into independent single-machine subproblems. Each subproblem is solved exactly by a dedicated single-machine AM scheduling oracle that combines batch-position branching, valid parallel and positional lower bounds, and dominance pruning. At the parallel-machine level, a node lower bound is obtained by combining oracle-based bounds on the assigned parts with per-part tardiness floors on the free parts, and a memoization pool reuses oracle values to both avoid recomputation and strengthen the node bound. The incumbent upper bound is produced by a multi-rule constructive heuristic refined by a move-based local search. Computational experiments show that the algorithm proves optimality for instances with up to 17 parts within seconds on the single machine, and that on the parallel-machine problem a corrected depth-first diving rule reduces both the search-tree size and the peak memory by up to several orders of magnitude relative to a balance-first rule, while leaving the proven optimum unchanged. We further report a negative result: a parallel positional free-part bound is structurally dominated by the per-part floor and yields no additional pruning.**

I. MATHEMATICAL PROGRAMMING MODEL

A. Problem Description

A set of parts $J = \{1, \ldots, n\}$ is to be processed on a set of identical parallel metal additive manufacturing machines $M = \{1, \ldots, \bar{M}\}$ based on Powder Bed Fusion technology. Without loss of generality, we assume that each customer order consists of a single part, so that order tardiness is equivalent to part tardiness. Each part $j \in J$ has a due date d_j, a volume v_j, and fixed geometric dimensions given by length l_j, width w_j, and height h_j. All machines are assumed to have identical rectangular build platforms, each with length L and width W.

Parts are assigned to machines and grouped into batches. On each machine, batches are processed sequentially from time zero, while different machines can process their assigned batches in parallel. We consider a one-dimensional capacity restriction, under which a set of parts can be assigned to the same batch if the sum of their projected areas does not exceed the platform capacity $L \times W$. This assumption gives rise to the identical parallel-machine one-dimensional batch scheduling problem studied in this paper.

The processing time of a batch consists of three components: the fixed machine setup time S, the laser scanning time, and the recoating time [7]. More specifically, the laser scanning time depends on the total volume of the parts assigned to the batch, whereas the recoating time depends on the maximum height of the parts in the batch. Let P_{mb} denote the processing time of batch position b on machine m, and let C_{mb} denote its completion time. If part j is assigned to batch position b on machine m, then its completion time c_j is equal to C_{mb}. The tardiness of part j is defined as

$$T_j = \max\{0,\, c_j - d_j\}.$$

All parts are assumed to be available at time zero. The objective is to determine the assignment of parts to machines, the grouping of parts into batches on each machine, and the sequence of batches on each machine so as to minimize the total tardiness of all parts.

B. MILP Formulation

We formulate the identical parallel-machine one-dimensional batch scheduling problem as a mixed-integer linear program (MILP). The formulation extends the single-machine model by introducing machine-indexed batch positions. Each part must be assigned to exactly one batch on exactly one machine, and batches on the same machine are processed sequentially from time zero. Since the machines are identical, the processing-time structure of each batch remains the same across machines.

Sets and Indices

- $J = \{1, \ldots, n\}$: Set of parts.
- $M = \{1, \ldots, \bar{M}\}$: Set of identical parallel machines.
- $B = \{1, \ldots, n\}$: Set of available batch positions on each machine.

Parameters

- l_j, w_j, h_j: Length, width, and height of part j.
- v_j: Volume of part j.
- d_j: Due date of part j.
- L, W: Length and width of the build platform.
- S: Setup time of a machine.
- V: Laser scanning time coefficient.
- U: Recoating time coefficient.
- $\mathcal{M}$: A sufficiently large positive constant.

Decision Variables

- x_{jmb}: equal to 1 if part j is assigned to batch position b on machine m, 0 otherwise.

This paragraph of the first footnote will contain the author affiliations, contact information, and email addresses in the final camera-ready version. It is currently blinded to comply with double-anonymous review guidelines.

This paragraph of the second footnote will contain funding information and grant numbers in the final camera-ready version. It is currently blinded to comply with double-anonymous review guidelines.

- z_{mb}: equal to 1 if batch position b on machine m is used, 0 otherwise.
- H_{mb}: height of batch b on machine m.
- P_{mb}: processing time of batch b on machine m.
- C_{mb}: completion time of batch b on machine m.
- c_j: completion time of part j.
- T_j: tardiness of part j.

The MILP formulation is given as follows:

$$\text{Minimize} \quad \sum_{j \in J} T_j \tag{1}$$

s.t.

$$\sum_{m \in M} \sum_{b \in B} x_{jmb} = 1, \quad \forall j \in J \tag{2}$$

$$\sum_{j \in J} x_{jmb} \leq \mathcal{M} z_{mb}, \quad \forall m \in M,\ \forall b \in B \tag{3}$$

$$\sum_{j \in J} (l_j w_j)\, x_{jmb} \leq LW, \quad \forall m \in M,\ \forall b \in B \tag{4}$$

$$H_{mb} \geq h_j x_{jmb}, \quad \forall j \in J,\ \forall m \in M,\ \forall b \in B \tag{5}$$

$$P_{mb} = S z_{mb} + V \sum_{j \in J} v_j x_{jmb} + U H_{mb}, \quad \forall m \in M,\ \forall b \in B \tag{6}$$

$$C_{m1} \geq P_{m1}, \quad \forall m \in M \tag{7}$$

$$C_{mb} \geq C_{m,b-1} + P_{mb}, \quad \forall m \in M,\ \forall b = 2, \ldots, |B| \tag{8}$$

$$c_j \geq C_{mb} - \mathcal{M}(1 - x_{jmb}), \quad \forall j \in J,\ \forall m \in M,\ \forall b \in B \tag{9}$$

$$T_j \geq c_j - d_j, \quad \forall j \in J \tag{10}$$

$$T_j \geq 0, \quad \forall j \in J \tag{11}$$

$$z_{m,b+1} \leq z_{mb}, \quad \forall m \in M,\ \forall b = 1, \ldots, |B| - 1 \tag{12}$$

$$x_{jmb}, z_{mb} \in \{0, 1\}, \quad H_{mb}, P_{mb}, C_{mb}, c_j, T_j \geq 0 \tag{13}$$

The objective function (1) minimizes the total tardiness of all parts. Constraints (2) ensure that each part is assigned to exactly one batch on exactly one machine. Constraints (3) activate batch position b on machine m whenever at least one part is assigned to it. Constraints (4) enforce the one-dimensional build-platform capacity restriction for every batch. Constraints (5) define the batch height as the maximum height of the parts assigned to that batch. Constraints (6) calculate the processing time of each batch. Constraints (7)–(8) define the sequential completion times of batches on each machine. Constraint (9) links each part completion time to the completion time of its assigned batch. Constraints (10)–(11) define part tardiness. Finally, constraints (12) enforce contiguous batch usage on each machine, i.e., batch position $b+1$ on machine m cannot be used unless batch position b on the same machine is used.

Because the machines are identical, the model also contains symmetry across machines. To mitigate this effect, an additional symmetry-breaking constraint may be introduced:

$$\sum_{b \in B} z_{mb} \geq \sum_{b \in B} z_{m+1,b}, \quad \forall m = 1, \ldots, \bar{M} - 1 \tag{14}$$

Constraint (14) imposes a nonincreasing order on the number of used batch positions across machines. In other words, machine (m+1) cannot use more batch positions than machine (m). Since the machines are identical, this restriction does not exclude any structurally distinct feasible schedule, but it helps reduce redundant symmetric solutions and can improve the tractability of the MILP.

II. METHODOLOGY

A. Design Rationale: Assignment-Based Branching versus Branch-and-Price

Two families of exact methods dominate the parallel-machine scheduling literature for min-sum objectives. The first is *assignment-based* (bin-packing-style) branch-and-bound: the search branches on the assignment of jobs to machines, and once an assignment is fixed the problem separates into independent single-machine subproblems. The second is *branch-and-price* (Dantzig–Wolfe / column generation), in which the master problem is a set-partitioning model whose columns are complete single-machine schedules and the pricing subproblem is a single-machine problem; this approach yields the strongest known relaxations and is the de facto state of the art for total (weighted) completion-time and tardiness objectives on parallel machines.

We deliberately adopt the assignment-based framework, and the choice is dictated by the structure of the single-machine subproblem in our setting. In classical parallel-machine problems the per-machine subproblem is polynomially solvable (e.g., WSPT for total weighted completion time, or EDD-type rules), so the per-machine evaluation at a leaf is closed-form. In the one-dimensional AM batch scheduling problem, by contrast, the single-machine subproblem is itself NP-hard: it couples an area-feasible (knapsack-type) batching decision with a sequencing decision and a batch processing time that depends nonlinearly on the total volume and the maximum height of the parts in each batch. We therefore embed an *exact single-machine oracle* at each leaf and memoize its values across the search tree (Section II-G), yielding a *nested* exact branch-and-bound in which the leaf evaluation is itself an exact search. This nested, oracle-driven evaluation is the main structural difference between our method and a textbook assignment-based branch-and-bound, whose leaves are evaluated in closed form.

The same hardness explains why we do not pursue branch-and-price here. In a column-generation formulation the pricing subproblem would be exactly this NP-hard, time-coupled prize-collecting batch-scheduling problem on a single machine; pricing would therefore be at least as hard as the oracle we already solve, and considerably more expensive per node. The assignment-based framework instead confines the hard sub-structure to an independent, memoizable single-machine oracle and exposes a cleanly separable node bound (Section II-D). Its two classical weaknesses—symmetry among identical machines, and weak lower bounds at shallow nodes where many parts are still unassigned—are addressed directly by the symmetry-reduced branching of Section II-C and the free-part bounds of Section II-D, respectively.

Closing the residual shallow-node gap is precisely where a column-generation bound would help, and we leave that heavier route to future work.

B. Overall Branch-and-Bound Framework

This section presents the overall exact solution framework for the identical parallel-machine AM batch scheduling problem. The proposed method follows a decomposition-oriented branch-and-bound structure. The upper-level search tree branches on part-to-machine assignments, while the detailed batching and sequencing decisions on each machine are evaluated by an exact single-machine scheduling oracle.

At the beginning of the algorithm, an initial feasible solution is constructed by a fast heuristic, which provides the initial global upper bound UB. The search is initialized at the root node N^0, where no part has been assigned to any machine:

$$\mathcal{S}_m(N^0) = \emptyset, \quad \forall m \in \mathcal{M},$$

and

$$\mathcal{U}(N^0) = \mathcal{J}.$$

The root node is inserted into the active node list.

During the search, an active node N is selected according to a node selection rule. The algorithm first evaluates a valid lower bound $LB(N)$. If

$$LB(N) \geq UB,$$

then node N cannot lead to a solution better than the incumbent and is pruned. Otherwise, the algorithm checks whether N represents a complete assignment. If

$$\mathcal{U}(N) = \emptyset,$$

then every part has been assigned to one machine. In this case, the problem decomposes into $|\mathcal{M}|$ independent single-machine scheduling subproblems. For each machine m, the exact oracle is called to solve the subproblem induced by the assigned part set $\mathcal{S}_m(N)$. The objective value of the complete assignment is then computed as

$$Z(N) = \sum_{m \in \mathcal{M}} \Phi\big(\mathcal{S}_m(N)\big),$$

where $\Phi(\mathcal{S}_m(N))$ denotes the optimal total tardiness of the single-machine problem associated with $\mathcal{S}_m(N)$. If $Z(N) < UB$, the incumbent solution and the global upper bound are updated.

If N is not a complete assignment and satisfies $LB(N) < UB$, the node is branched by selecting one unassigned part $j \in \mathcal{U}(N)$ and assigning it to one of the parallel machines. Each resulting child node fixes one additional part-to-machine assignment. For each generated child node, the lower bound is evaluated, and the child node is inserted into the active node list only if its lower bound is smaller than the current upper bound.

The algorithm terminates when the active node list becomes empty. At this point, every unexplored node has either been evaluated exactly or pruned by a valid lower bound. Therefore, the incumbent solution is guaranteed to be globally optimal.

The overall procedure is summarized as follows.

1) Construct an initial feasible solution and set its objective value as the initial upper bound UB.
2) Initialize the root node N^0 with $\mathcal{S}_m(N^0) = \emptyset$ for all $m \in \mathcal{M}$ and $\mathcal{U}(N^0) = \mathcal{J}$. Insert N^0 into the active node list $\mathcal{L}$.
3) While $\mathcal{L}$ is not empty, select a node N from $\mathcal{L}$ according to the node selection rule.
4) Compute a valid lower bound $LB(N)$. If $LB(N) \geq UB$, prune node N.
5) If $\mathcal{U}(N) = \emptyset$, evaluate the complete assignment by solving the single-machine subproblem induced by each $\mathcal{S}_m(N)$ using the exact oracle. Compute
$$Z(N) = \sum_{m \in \mathcal{M}} \Phi\big(\mathcal{S}_m(N)\big).$$
If $Z(N) < UB$, update the incumbent solution and set $UB \leftarrow Z(N)$.
6) If $\mathcal{U}(N) \neq \emptyset$ and $LB(N) < UB$, select a branching part $j \in \mathcal{U}(N)$ and generate child nodes by assigning j to candidate machines.
7) For each generated child node N', compute or update $LB(N')$. If $LB(N') < UB$, insert N' into $\mathcal{L}$; otherwise, prune N'.
8) When $\mathcal{L}$ becomes empty, return the incumbent solution and its objective value UB.

C. Assignment-Based Search Tree

The upper-level search tree is constructed over part-to-machine assignments. Instead of explicitly branching on batch compositions and batch sequences, these machine-internal decisions are deferred to the single-machine oracle. Therefore, each node in the upper-level tree records only which parts have already been fixed to each machine and which parts remain unassigned.

1) Node Representation: Let $\mathcal{M} = \{1, \ldots, M\}$ denote the set of identical parallel machines and let $\mathcal{J} = \{1, \ldots, n\}$ denote the set of parts. For a node N, let

$$\mathcal{S}_m(N) \subseteq \mathcal{J}, \qquad m \in \mathcal{M},$$

be the set of parts that have already been assigned to machine m. Let

$$\mathcal{U}(N) = \mathcal{J} \setminus \bigcup_{m \in \mathcal{M}} \mathcal{S}_m(N)$$

be the set of parts that remain unassigned at node N.

Thus, node N is represented as

$$N = \big(\mathcal{S}_1(N), \ldots, \mathcal{S}_M(N), \mathcal{U}(N)\big),$$

where the assigned sets are mutually disjoint, i.e.,

$$\mathcal{S}_m(N) \cap \mathcal{S}_{m'}(N) = \emptyset, \qquad \forall m \neq m'.$$

The root node N^0 is defined by

$$\mathcal{S}_m(N^0) = \emptyset, \quad \forall m \in \mathcal{M}, \qquad \mathcal{U}(N^0) = \mathcal{J}.$$

A node is called terminal if all parts have been assigned, namely $\mathcal{U}(N) = \emptyset$. At a terminal node, the parallel-machine problem decomposes into M independent single-machine subproblems. The exact single-machine oracle is then called for each assigned set $\mathcal{S}_m(N)$ to evaluate the complete assignment.

2) Symmetry-Reduced Branching Scheme: At a nonterminal node N, one unassigned part $j \in \mathcal{U}(N)$ is selected for branching. A straightforward branching rule would assign part j to any machine $m \in \mathcal{M}$. However, because the machines are identical, this rule creates many symmetric nodes. For example, assigning the first part to machine 1 or machine 2 yields equivalent partial assignments up to a relabeling of machines.

To reduce such symmetry, we impose a canonical machine-opening rule. At any node N, let

$$q(N) = |\{m \in \mathcal{M} : \mathcal{S}_m(N) \neq \emptyset\}|$$

denote the number of machines that have already been used. Machines are opened in their index order. Therefore, the selected part j can be assigned either to one of the already used machines or to the first unused machine. The candidate machine set is defined as

$$\mathcal{K}(N) = \begin{cases} \{1\}, & q(N) = 0, \\ \{1, \ldots, \min\{q(N)+1, M\}\}, & q(N) \geq 1. \end{cases}$$

For each candidate machine $r \in \mathcal{K}(N)$, a child node $N^{(r)}$ is generated by assigning part j to machine r:

$$\mathcal{S}_r\big(N^{(r)}\big) = \mathcal{S}_r(N) \cup \{j\},$$

$$\mathcal{S}_m\big(N^{(r)}\big) = \mathcal{S}_m(N), \qquad \forall m \in \mathcal{M},\ m \neq r,$$

and

$$\mathcal{U}\big(N^{(r)}\big) = \mathcal{U}(N) \setminus \{j\}.$$

This branching rule preserves completeness. Since the machines are identical, any feasible assignment that uses a later unused machine before an earlier unused machine has an equivalent assignment obtained by renaming machines. Therefore, restricting the search to the canonical machine-opening order does not remove any structurally distinct feasible solution. It only eliminates redundant symmetric representations.

3) Branching Part Selection: The efficiency of the assignment-based search depends strongly on the choice of the part selected for branching. A simple rule is to select the unassigned part with the earliest due date, since such parts are usually more critical for the total tardiness objective. Another possible rule is to select the part with the largest projected area, volume, or height, because such parts have a stronger impact on batching feasibility and processing times.

In this study, we use a lower-bound-based branching score to select the branching part. The idea is to choose a part whose assignment to different machines leads to the largest variation in the lower bounds of the child nodes. Such a part is more informative for the search because it can better differentiate promising and unpromising assignment decisions.

For each candidate part $j \in \mathcal{U}(N)$ and each candidate machine $r \in \mathcal{K}(N)$, let $N_j^{(r)}$ denote the child node obtained by assigning j to machine r. A tentative lower bound $LB(N_j^{(r)})$ is computed or estimated for each child node. The branching score of part j is defined as

$$\text{score}(j) = \max_{r \in \mathcal{K}(N)} LB\big(N_j^{(r)}\big) - \min_{r \in \mathcal{K}(N)} LB\big(N_j^{(r)}\big).$$

The selected branching part is then

$$j^* = \arg\max_{j \in \mathcal{U}(N)} \text{score}(j).$$

This rule favors parts whose machine assignment has a large impact on the node lower bound. As a result, the generated child nodes tend to have more distinct lower bounds, which improves the ability of the branch-and-bound algorithm to identify and prune inferior branches.

To reduce the computational overhead of evaluating all unassigned parts, a restricted version can also be used. In this variant, the score is computed only for a candidate subset $\mathcal{P}(N) \subseteq \mathcal{U}(N)$, such as the parts with the earliest due dates or the largest projected areas. The branching part is then selected as

$$j^* = \arg\max_{j \in \mathcal{P}(N)} \text{score}(j).$$

This restricted strong-branching strategy provides a balance between branching quality and computational efficiency. "'

D. Node Lower Bound

This subsection presents the lower-bound evaluation used for partial assignment nodes. Since the upper-level search tree fixes only part-to-machine assignments, while the batching and sequencing decisions on each machine are deferred to the single-machine oracle, the lower bound is constructed at the machine-set level. The bound consists of three components: a machine-wise monotonicity bound, a memory-based bound obtained from previous oracle calls, and a fast analytical bound. An additional relaxation is further introduced to account for the unassigned parts.

1) Machine-Wise Monotonicity Bound: For any part subset $\mathcal{Q} \subseteq \mathcal{J}$, let $\Phi(\mathcal{Q})$ denote the optimal total tardiness of the corresponding single-machine problem in which all parts in $\mathcal{Q}$ are processed on one machine starting from time zero. The following monotonicity property is used to construct a valid lower bound for partial assignment nodes.

Proposition 1. For any two part subsets $\mathcal{Q}_1$ and $\mathcal{Q}_2$ satisfying $\mathcal{Q}_1 \subseteq \mathcal{Q}_2 \subseteq \mathcal{J}$, we have

$$\Phi(\mathcal{Q}_1) \leq \Phi(\mathcal{Q}_2).$$

Proof. Consider an optimal single-machine schedule for $\mathcal{Q}_2$. By removing all parts in $\mathcal{Q}_2 \setminus \mathcal{Q}_1$ from this schedule and keeping the relative order and batch membership of the remaining parts, we obtain a feasible schedule for $\mathcal{Q}_1$. Removing parts cannot increase the processing time of any remaining batch, because the volume component is reduced and the maximum height of a batch cannot increase.

Therefore, the completion time of each part in $\mathcal{Q}_1$ does not increase. Since part tardiness is nondecreasing in its completion time, the total tardiness of the remaining parts is no larger than their contribution in the original schedule for $\mathcal{Q}_2$. Moreover, the parts in $\mathcal{Q}_2 \setminus \mathcal{Q}_1$ have nonnegative tardiness. Hence, the optimal value for $\mathcal{Q}_1$ cannot exceed that for $\mathcal{Q}_2$, which proves the proposition. □

For a partial assignment node N, the set $\mathcal{S}_m(N)$ contains the parts that have already been fixed to machine m. In any descendant of node N, machine m must process at least these parts, although additional unassigned parts may later be assigned to the same machine. By Proposition 1, the optimal total tardiness contribution of machine m in any completion of node N is at least $\Phi(\mathcal{S}_m(N))$. Therefore, the following basic node lower bound is valid:

$$LB^{\mathrm{mono}}(N) = \sum_{m\in\mathcal{M}} \Phi\big(\mathcal{S}_m(N)\big).$$

This bound is strong when the exact values $\Phi(\mathcal{S}_m(N))$ are available. However, computing them exactly at every node would require a large number of calls to the single-machine oracle. Therefore, practical lower bounds are introduced below.

2) Memory-Based Lower Bound: During the search, some single-machine subproblems are solved exactly by the oracle, especially when complete assignments are evaluated or when a machine-wise subset is selected for exact evaluation. These solved subsets and their optimal values are stored in a memory pool. Let $\mathcal{C}$ denote the collection of part subsets whose exact single-machine optimal values have been computed. For each $\mathcal{Q} \in \mathcal{C}$, the value $\Phi(\mathcal{Q})$ is stored.

For a given assigned set $\mathcal{S}_m(N)$, the memory-based lower bound is defined as

$$LB_m^{\mathrm{mem}}\big(\mathcal{S}_m(N)\big) = \max\left\{\Phi(\mathcal{Q}) : \mathcal{Q} \in \mathcal{C},\ \mathcal{Q} \subseteq \mathcal{S}_m(N)\right\}.$$

If no stored subset is contained in $\mathcal{S}_m(N)$, the value of $LB_m^{\mathrm{mem}}(\mathcal{S}_m(N))$ is set to zero.

The validity of this bound follows directly from Proposition 1. For any stored subset $\mathcal{Q} \subseteq \mathcal{S}_m(N)$, we have

$$\Phi(\mathcal{Q}) \le \Phi\big(\mathcal{S}_m(N)\big).$$

Taking the maximum over all such stored subsets therefore yields a valid lower bound on $\Phi(\mathcal{S}_m(N))$. The corresponding memory-based node lower bound is

$$LB^{\mathrm{mem}}(N) = \sum_{m\in\mathcal{M}} LB_m^{\mathrm{mem}}\big(\mathcal{S}_m(N)\big).$$

This mechanism allows the algorithm to reuse information obtained from previous oracle calls. As the search proceeds, the memory pool becomes richer, and the memory-based bound can become increasingly effective for pruning nodes whose assigned sets contain previously solved difficult subsets.

3) Fast Analytical Lower Bound: In addition to the memory-based bound, we compute a fast analytical lower bound for each machine-wise assigned set. For a node N and a machine $m \in \mathcal{M}$, let

$$\mathcal{Q}_m(N) = \mathcal{S}_m(N)$$

denote the set of parts that have already been assigned to machine m. The purpose of this bound is to estimate $\Phi(\mathcal{Q}_m(N))$ without invoking the exact single-machine oracle.

If $\mathcal{Q}_m(N) = \emptyset$, we set

$$LB_m^{\mathrm{fast}}(\mathcal{Q}_m(N)) = 0.$$

Otherwise, two analytical lower bounds are evaluated.

The first one is the parallel lower bound. It relaxes the single-machine sequencing restriction by assuming that each part in $\mathcal{Q}_m(N)$ can be processed independently as a singleton batch starting from time zero. Thus,

$$LB_m^{\mathrm{par}}(\mathcal{Q}_m(N)) = \sum_{j\in\mathcal{Q}_m(N)} \max\left\{0,\ S + Vv_j + Uh_j - d_j\right\}.$$

The second one is the positional lower bound. Let $q_m = |\mathcal{Q}_m(N)|$. Sort the parts in $\mathcal{Q}_m(N)$ by projected area, height, volume, and due date, respectively:

$$a_{(1)} \le \cdots \le a_{(q_m)}, \qquad h_{[1]} \le \cdots \le h_{[q_m]},$$
$$v_{\langle 1\rangle} \le \cdots \le v_{\langle q_m\rangle}, \qquad d^{\uparrow}_{[1]} \le \cdots \le d^{\uparrow}_{[q_m]}.$$

For each position $k = 1, \ldots, q_m$, define

$$A_k = \sum_{r=1}^{k} a_{(r)}, \qquad \beta_k = \left\lceil \frac{A_k}{LW} \right\rceil,$$
$$\underline{V}_k = \sum_{r=1}^{k} v_{\langle r\rangle}, \qquad \underline{H}_k = \sum_{r=1}^{\beta_k - 1} h_{[r]} + h_{[k]}.$$

Then the completion time of the k-th completed part is lower bounded by

$$\underline{C}_k = \beta_k S + V\underline{V}_k + U\underline{H}_k.$$

The positional lower bound is computed as

$$LB_m^{\mathrm{pos}}(\mathcal{Q}_m(N)) = \sum_{k=1}^{q_m} \max\left\{0,\ \underline{C}_k - d^{\uparrow}_{[k]}\right\}.$$

The derivation and validity proof of this bound are given in Appendix .

The fast analytical lower bound is defined as

$$LB_m^{\mathrm{fast}}(\mathcal{Q}_m(N)) = \max\left\{LB_m^{\mathrm{par}}(\mathcal{Q}_m(N)),\ LB_m^{\mathrm{pos}}(\mathcal{Q}_m(N))\right\}.$$

Since both $LB_m^{\mathrm{par}}(\mathcal{Q}_m(N))$ and $LB_m^{\mathrm{pos}}(\mathcal{Q}_m(N))$ are valid lower bounds on $\Phi(\mathcal{Q}_m(N))$, their maximum is also valid:

$$LB_m^{\mathrm{fast}}(\mathcal{Q}_m(N)) \le \Phi(\mathcal{Q}_m(N)).$$

The machine-wise lower bound used in the assignment search is then

$$LB_m(\mathcal{Q}_m(N)) = \max\left\{LB_m^{\mathrm{mem}}(\mathcal{Q}_m(N)),\ LB_m^{\mathrm{fast}}(\mathcal{Q}_m(N)),\ LB_m^{\mathrm{heavy}}(\mathcal{Q}\right.$$

where the heavy-machine term LB_m^{heavy} is defined in Section II-D.4 and is zero on lightly loaded machines. Therefore, the assigned-part lower bound of node N is

$$LB^{\text{ass}}(N) = \sum_{m \in \mathcal{M}} LB_m(\mathcal{Q}_m(N)).$$

4) Heavy-Machine Oracle Bound: The analytical bound LB_m^{fast} rests on a positional argument that pairs sorted positional completions with sorted due dates. On a heavily loaded machine—one carrying many parts—this pairing is very loose and can collapse to zero even when the true single-machine tardiness Φ is large, because it optimistically assigns the tight-due-date parts to the earliest positions, a configuration a congested machine cannot realize. Such heavily loaded machines occur only in extremely unbalanced complete assignments, which are almost never optimal; yet without a tight bound the search descends into these subtrees and pays the full cost of an exact oracle evaluation on a large part set.

To prune these subtrees, we compute a stronger machine-wise bound whenever a machine is heavily loaded. Concretely, when $|\mathcal{Q}_m(N)| \geq K$ with $K = \lceil n/\bar{M} \rceil + 4$ (a load exceeding the balanced load $\lceil n/\bar{M} \rceil$ by a fixed margin), we run the single-machine oracle on $\mathcal{Q}_m(N)$ with a small budget of n^2 expanded nodes and take its *proven* lower bound $LB_m^{\text{heavy}}(\mathcal{Q}_m(N))$, namely the minimum node bound over its unexplored frontier when the budget is reached (the exact Φ if it finishes first). Since any valid lower bound preserves correctness, this only tightens the node bound; it is memoized per part set, and is set to zero on machines below the threshold. Both the threshold K and the budget n^2 scale automatically with the instance, so no manual tuning is required. On a 20-part instance this bound lifts heavily loaded machines whose positional bound is essentially zero to their true tardiness of 20–30, well above the incumbent, pruning the corresponding subtrees before any large oracle call.

5) Unassigned-Part Relaxation: The bound $LB^{\text{ass}}(N)$ accounts only for the parts that have already been assigned to machines. At early nodes, this bound can be weak because many parts remain unassigned. To strengthen the node evaluation, we add a relaxation for the unassigned parts.

For each unassigned part $j \in \mathcal{U}(N)$, its completion time in any feasible schedule cannot be earlier than the processing time of a singleton batch containing only part j and starting at time zero. This singleton processing time is

$$p_j^{\text{sing}} = S + Vv_j + Uh_j.$$

Therefore, the tardiness contribution of part j is at least

$$\max\left\{0,\ p_j^{\text{sing}} - d_j\right\}$$

under this relaxation. Summing over all unassigned parts yields

$$LB^{\text{U}}(N) = \sum_{j \in \mathcal{U}(N)} \max\left\{0,\ S + Vv_j + Uh_j - d_j\right\}.$$

This is a valid lower bound on the total tardiness contribution of the unassigned parts. The relaxation is optimistic because it allows every unassigned part to be processed independently as a singleton batch starting from time zero, without considering machine capacity, machine occupation, or competition among unassigned parts.

Combining the assigned-part bound and the unassigned-part relaxation, the final node lower bound is defined as

$$LB(N) = LB^{\text{ass}}(N) + LB^{\text{U}}(N).$$

Equivalently,

$$LB(N) = \sum_{m \in \mathcal{M}} LB_m\big(\mathcal{S}_m(N)\big) + \tag{15}$$

$$\sum_{j \in \mathcal{U}(N)} \max\left\{0,\ S + Vv_j + Uh_j - d_j\right\}. \tag{16}$$

The validity of $LB(N)$ follows from the fact that the first term is a lower bound on the tardiness contribution of all parts already assigned to machines, while the second term is a lower bound on the tardiness contribution of the remaining unassigned parts. Since these two sets of parts are disjoint, their lower bounds can be added. A node N can therefore be safely pruned whenever

$$LB(N) \geq UB,$$

where UB is the incumbent upper bound.

E. Upper Bound and Complete Assignment Evaluation

An incumbent upper bound is required to prune nodes during the branch-and-bound search. In the proposed framework, an upper bound is obtained from any complete feasible assignment of parts to machines, because the batching and sequencing decisions on each machine can then be evaluated exactly by the single-machine oracle.

1) Initial Upper Bound: At the beginning of the search, a constructive heuristic is used to generate an initial complete assignment. The heuristic first sorts all parts in nondecreasing order of due dates. The parts are then assigned one by one to the parallel machines. When considering a part j, the heuristic evaluates the effect of assigning j to each candidate machine and selects the machine that yields the smallest estimated increase in the total tardiness.

Let $\widehat{\Phi}(\mathcal{Q})$ denote a fast heuristic estimate of the single-machine total tardiness for processing the part set $\mathcal{Q}$ on one machine. For the current partial assignment $\{\mathcal{S}_m\}_{m \in \mathcal{M}}$, the selected machine for part j is given by

$$r^* = \arg\min_{r \in \mathcal{M}} \left\{\widehat{\Phi}(\mathcal{S}_r \cup \{j\}) - \widehat{\Phi}(\mathcal{S}_r)\right\}.$$

Part j is then assigned to machine r^*, and the assigned set of that machine is updated as

$$\mathcal{S}_{r^*} \leftarrow \mathcal{S}_{r^*} \cup \{j\}.$$

After all parts have been assigned, the resulting complete assignment is evaluated exactly. For each machine $m \in \mathcal{M}$,

the exact single-machine oracle is called to compute $\Phi(\mathcal{S}_m)$. The initial upper bound is then set as

$$UB = \sum_{m \in \mathcal{M}} \Phi(\mathcal{S}_m).$$

The corresponding assignment and the associated single-machine schedules are stored as the initial incumbent solution.

In practice, several constructive rules can be used to generate multiple initial assignments, such as earliest due date, largest projected area, largest volume, and largest height. Each complete assignment is evaluated exactly by the oracle, and the best one is selected as the initial incumbent. This strategy increases the chance of obtaining a tight initial upper bound while requiring only a limited number of oracle calls.

2) Local-Search Refinement: The constructive assignment is further improved by a move-based local search before the branch-and-bound search starts. Let $\mathcal{A} = \{\mathcal{S}_m\}_{m \in \mathcal{M}}$ be the current complete assignment. A *move* relocates a single part j from its current machine m to another machine m'. Because the objective is separable across machines once the assignment is fixed, the change in objective induced by such a move is

$$\Delta(j, m \to m') = \Big[\Phi(\mathcal{S}_m \setminus \{j\}) + \Phi(\mathcal{S}_{m'} \cup \{j\})\Big] - \Big[\Phi(\mathcal{S}_m) + \Phi(\mathcal{S}_{m'})\Big],$$

and depends only on the two affected machines. The search repeatedly applies the first move with $\Delta < 0$ and terminates when no improving move remains. All four oracle evaluations in Δ are served from, or added to, the memory pool $\mathcal{C}$, so the refinement is inexpensive.

The local search only ever decreases the incumbent value, so it can never invalidate the upper bound or remove an optimal solution; it merely tightens UB before the search. A second, more subtle effect arises through the memory pool. Each oracle call issued while evaluating a move stores the value $\Phi(\mathcal{S}_{m'} \cup \{j\})$ of a configuration that consists of the parts already assigned to a machine together with one foreign part. These are exactly the machine subsets that the assignment-based search visits when it branches a free part onto a loaded machine. Consequently, the move evaluations *warm* the memory pool with configurations whose stored oracle values later sharpen the memory-based machine lower bound of Section II-D, tightening node bounds during the search in addition to lowering UB. Both effects are correctness-preserving: the former only lowers a valid upper bound, the latter only adds valid lower-bound witnesses to the pool.

3) Complete Assignment Evaluation: A node N is a complete assignment if

$$\mathcal{U}(N) = \emptyset.$$

At such a node, every part has been assigned to exactly one machine. Therefore, the remaining optimization decisions are independent across machines. For each machine m, the assigned part set $\mathcal{S}_m(N)$ defines a single-machine AM batch scheduling subproblem.

The exact value of the complete assignment is computed as

$$Z(N) = \sum_{m \in \mathcal{M}} \Phi\big(\mathcal{S}_m(N)\big),$$

where $\Phi(\mathcal{S}_m(N))$ is obtained by the exact single-machine oracle. If $\mathcal{S}_m(N) = \emptyset$, then

$$\Phi\big(\mathcal{S}_m(N)\big) = 0.$$

To avoid repeated computations, all oracle evaluations are stored in the memory pool $\mathcal{C}$. Before calling the oracle for a set $\mathcal{S}_m(N)$, the algorithm first checks whether this set has already been evaluated. If $\mathcal{S}_m(N) \in \mathcal{C}$, the stored value $\Phi(\mathcal{S}_m(N))$ is directly retrieved. Otherwise, the oracle is called, and the computed value is added to the memory pool.

Incremental lower-bound-guarded evaluation.: Evaluating $\Phi(\mathcal{S}_m(N))$ for the most heavily loaded machine is by far the most expensive oracle call at a leaf. To avoid it whenever possible, the machines are evaluated one at a time in nondecreasing order of $|\mathcal{S}_m(N)|$ (smallest part set first). Let $\mathcal{E}$ be the set of machines already evaluated exactly and $\overline{\mathcal{E}} = \mathcal{M} \setminus \mathcal{E}$ the remaining ones. Before each oracle call, the algorithm tests the valid running bound

$$\sum_{m \in \mathcal{E}} \Phi\big(\mathcal{S}_m(N)\big) + \sum_{m \in \overline{\mathcal{E}}} LB_m\big(\mathcal{S}_m(N)\big) \;\geq\; UB,$$

where $LB_m(\cdot)$ is the machine-wise lower bound of Section II-D.3. If the inequality holds, the leaf cannot improve the incumbent and is pruned immediately, so the remaining (typically largest and most costly) oracle calls are skipped. Because the test combines exact values for the evaluated machines with valid lower bounds for the rest, it never discards an improving leaf; it only avoids provably useless exact computations.

Once $Z(N)$ is obtained, it is compared with the current incumbent upper bound. If

$$Z(N) < UB,$$

then the incumbent solution is updated and the upper bound is reset as

$$UB \leftarrow Z(N).$$

Otherwise, the complete assignment is discarded.

Because each leaf node is evaluated by an exact single-machine oracle, the value $Z(N)$ is the true objective value of the corresponding complete parallel-machine assignment. Consequently, the incumbent solution maintained by the algorithm is always feasible for the original problem, and its value is a valid upper bound throughout the search.

F. Node Selection and Pruning

The active nodes generated during the assignment-based search are stored in an active node list, denoted by $\mathcal{L}$. The order in which nodes are selected from $\mathcal{L}$ affects both the speed of finding high-quality incumbent solutions and the efficiency of proving optimality. Therefore, the proposed algorithm adopts a hybrid node selection strategy that combines a load-balanced diving rule and best-first search.

1) Node Selection Strategy: A best-first strategy selects the active node with the smallest lower bound:

$$N^* = \arg\min_{N \in \mathcal{L}} LB(N).$$

This strategy is useful for improving the global lower bound and proving optimality. However, in the early stage of the search, best-first selection may delay the discovery of complete assignments and may therefore provide weak incumbent upper bounds.

A diving strategy, by contrast, descends quickly toward complete assignments. Let $d(N)$ denote the depth of node N, namely the number of parts already assigned, and let

$$\ell(N) = \max_{m \in \mathcal{M}} \big|\mathcal{S}_m(N)\big|$$

denote the load of its most heavily loaded machine. the algorithm selects the deepest active node and breaks ties toward the most balanced one,

$$N^* = \arg\max_{N \in \mathcal{L}} d(N), \qquad \text{ties broken by } \arg\min_{N} \ell(N).$$

Because each branching step assigns one more part, always expanding a deepest node descends to a complete assignment in $O(n)$ selections, so a feasible incumbent is obtained quickly. The tie-break toward the most balanced node keeps the leaf reached along the dive balanced, so its complete assignment tends to be near-optimal and every machine subset evaluated by the oracle stays small, which keeps the leaf evaluations cheap. Being a genuine depth-first descent, the dive expands one node and enqueues only its children, so the active node list stays bounded; this is what makes the diving mode effective for controlling memory. A rule that instead prioritizes the most balanced node over depth does not descend: it sweeps the frontier by load level and lets the active list grow, which we found ineffective both for obtaining incumbents and for bounding memory (Section IV-C).

To balance these two effects, we use a hybrid node selection strategy. Although an initial incumbent is available before the search starts, the algorithm first uses a balanced diving phase to improve the incumbent quickly. It then switches to best-first search to accelerate the proof of optimality. In addition, two thresholds, $N_{\max}$ and $N_{\min}$, are used to control the size of the active node list. If the number of active nodes exceeds $N_{\max}$, the algorithm switches to the diving mode to reduce memory consumption. Once the number of active nodes decreases below $N_{\min}$, the algorithm switches back to best-first mode.

More specifically, the node selection rule is defined as follows:

$$N^* = \begin{cases} \arg\max_{N \in \mathcal{L}} d(N), & \text{if diving mode is active,} \\ \arg\min_{N \in \mathcal{L}} LB(N), & \text{if best-first mode is active.} \end{cases}$$

In best-first mode, ties in $LB(N)$ are broken by larger depth; in diving mode, ties in depth are broken by selecting the most balanced node, i.e. the one with the smallest $\ell(N)$. Both tie-breaks steer the search toward complete, balanced assignments.

2) Pruning Rules: Several pruning rules are applied to reduce the search space.

Bound-based pruning.: Let UB denote the current incumbent upper bound. For any active node N, if

$$LB(N) \geq UB,$$

then no descendant of N can improve the incumbent solution. Therefore, node N is safely pruned.

Terminal-node evaluation.: If node N is terminal, namely

$$\mathcal{U}(N) = \emptyset,$$

then all parts have been assigned to machines. The node is evaluated exactly by calling the single-machine oracle for each machine:

$$Z(N) = \sum_{m \in \mathcal{M}} \Phi\big(\mathcal{S}_m(N)\big).$$

If $Z(N) < UB$, the incumbent solution is updated and UB is set to $Z(N)$. After this evaluation, the terminal node is discarded because it has no children.

Cache-based pruning and reuse.: Before evaluating a machine-wise subset $\mathcal{S}_m(N)$ by the exact single-machine oracle, the algorithm checks whether this subset has already been solved and stored in the memory pool $\mathcal{C}$. If $\mathcal{S}_m(N) \in \mathcal{C}$, the stored value $\Phi(\mathcal{S}_m(N))$ is retrieved directly. Otherwise, the oracle is called and the resulting value is added to $\mathcal{C}$. This mechanism does not alter the search space, but it avoids repeated exact evaluations and strengthens the memory-based lower bound for subsequent nodes.

Symmetry-based pruning.: The symmetry-reduced branching scheme described in Section II-C prevents the generation of nodes that differ only by a relabeling of identical machines. In particular, a new machine can be opened only after all earlier machine indices have already been opened. As a result, symmetric assignment patterns are removed before entering the active node list.

3) Optimality Guarantee: The proposed pruning rules do not remove any node that may contain a solution better than the incumbent. Bound-based pruning is valid because $LB(N)$ is a lower bound on the objective value of all complete assignments descending from node N. Terminal nodes are evaluated exactly using the single-machine oracle. The symmetry-reduced branching rule preserves at least one representative for every class of equivalent assignments under machine relabeling. Therefore, when the active node list becomes empty, all nonpruned complete assignments have been evaluated and all pruned nodes have been proven unable to improve the incumbent. The final incumbent solution is therefore globally optimal.

G. Exact Single-Machine Scheduling Oracle

In the proposed assignment-based branch-and-bound framework, each terminal node fixes a complete assignment of parts to parallel machines. Once the assignment is fixed, the problem decomposes into independent single-machine AM batch scheduling subproblems. To evaluate

each subproblem exactly, we use a dedicated single-machine scheduling oracle.

For a given part subset $\mathcal{Q} \subseteq \mathcal{J}$, the oracle solves the following problem: determine the batching and sequencing decisions on a single AM machine so as to minimize the total tardiness of the parts in $\mathcal{Q}$. The optimal objective value returned by the oracle is denoted by

$$\Phi(\mathcal{Q}).$$

If $\mathcal{Q} = \emptyset$, we set $\Phi(\mathcal{Q}) = 0$.

The oracle is implemented as an exact branch-and-bound algorithm. Each node represents a partial batch sequence fixed from time zero and a set of remaining unscheduled parts. Child nodes are generated by selecting a feasible subset of the remaining parts as the next batch. The search is accelerated by a constructive upper bound, valid lower bounds, adaptive node selection, and dominance-based pruning. Since the branching scheme enumerates all feasible batch sequences and pruning is performed only by valid lower bounds and dominance rules, the oracle returns the globally optimal value $\Phi(\mathcal{Q})$ whenever it terminates with an empty active node list.

All oracle calls are cached. Therefore, if the same subset $\mathcal{Q}$ is encountered again during the parallel-machine search, the stored value $\Phi(\mathcal{Q})$ is retrieved directly without resolving the subproblem. The detailed oracle implementation is provided in Appendix .

III. Test Instances

The computational experiments are conducted on test instances adopted from Mao et al. [3]. For each part j, the due date d_j is generated following the procedure of Yu et al. [10]. Specifically, due dates are sampled from a uniform distribution:

$$d_j \sim \text{Unif}\big[\mu(1 - TF - RDD/2),\ \mu(1 - TF + RDD/2)\big] \tag{17}$$

where TF and RDD denote the tardiness factor and the relative due-date range, respectively. A larger value of TF corresponds to tighter due dates and therefore a more time-critical demand setting. The parameter μ represents an estimate of the makespan of the instance. For the single-machine setting considered here, μ is estimated by a simple first-fit heuristic in which parts are sorted in non-decreasing order of projected area. In these instances, both part dimensions range from 10 to 19, and the machine platform size is 20×20.

IV. Computational Results

The algorithm was implemented in C++ and compiled with `g++ -O2`. All experiments were run single-threaded on a common workstation. Unless stated otherwise, the default configuration is used: the single-machine oracle uses parallel and positional lower bounds with shallow-node gating, adjacent-batch interchange dominance, and adaptive node selection; the parallel-machine search uses the strong oracle as Φ, the local-search upper bound of Section II-E.2, and the memoization pool. Reported objective values are total tardiness.

TABLE I

MILP solve effort under Gurobi for the idealized p-batch model versus our mix-batch model (same instances, $\bar{M} = 2$, 60 s limit). Both are proven optimal except †, which hits the time limit.

Instance	n	p-batch		mix-batch		$\frac{t_{\text{mix}}}{t_{\text{pb}}}$
		t(s)	nodes	t(s)	nodes	
5part	5	0.01	1	0.01	1	1×
10part	10	0.16	1	4.45	62,430	28×
11part	11	0.08	1	2.68	32,517	34×
12part	12	0.19	742	6.68	70,481	35×
13part	13	0.34	1	41.78	346,500	123×
14part	14	0.05	1	60.0†	333,206	>1200×

A. MILP Tractability: mix-batch versus idealized p-batch

We first quantify why a tailored exact method is needed at all, by measuring how the batch processing-time structure affects the difficulty of solving the problem with a general-purpose MILP solver (Gurobi). We use a single position-indexed MILP and toggle *only* the batch processing-time definition, holding the instances, machine count, due dates, area capacity, objective, and solver settings fixed. Two models are compared: our realistic *mix-batch* model, $P(B) = S + V\sum_{j\in B} v_j + U\max_{j\in B} h_j$, in which the laser-scan component is serial (additive in volume) while recoating is parallel (governed by the maximum height); and the idealized *p-batch* (parallel-batch) model used in prior AM scheduling work, $P(B) = S + \max_{j\in B}(Vv_j + Uh_j)$, in which the longest single job governs the whole batch. The two models coincide on singleton batches and differ only in how multi-part batches accumulate: a *sum* of volumes (serial) versus a *max* (parallel). All runs use $\bar{M} = 2$ and a 60 s time limit.

The contrast is stark (Table I). The p-batch model is essentially trivial for the solver: it is solved at the *root node* (one node, no branching) in under one second for every instance up to $n = 14$, because the parallel-batch structure makes consolidating parts into a batch almost free and the LP relaxation nearly integral. The mix-batch model, by contrast, requires tens to hundreds of thousands of branch-and-bound nodes; its solve time grows from 4.5 s at $n = 10$ to 41.8 s at $n = 13$, and at $n = 14$ it is *not* solved within 60 s (optimality gap 25.6%). The solve-time ratio $t_{\text{mix}}/t_{\text{pb}}$ rises from 28× to over 1200×. The additive volume term is what makes the difference: every part added to a batch elongates it and delays all downstream batches, which couples the batching and sequencing decisions and weakens the relaxation.

Two consequences follow. First, the idealized p-batch model is not merely easier but a *different* problem: its optimal tardiness is roughly half that of the mix-batch model on the same instances (e.g., 25.10 versus 59.47 on `13part`, 23.03 versus 45.63 on `10part`), so scheduling with the p-batch idealization systematically underestimates tardiness and yields optimistic, infeasible-in-practice decisions. Second, since the realistic mix-batch model is intractable for an off-the-shelf MILP already at $n = 14$ on two machines, a

TABLE II

SINGLE-MACHINE ORACLE ON BENCHMARK INSTANCES. "POPPED" IS THE NUMBER OF EXPANDED NODES; ALL INSTANCES ARE PROVEN OPTIMAL.

Instance	n	Total tardiness	Popped	Time (s)
5part	5	40.5691	26	0.000
10part	10	116.30	6,192	0.010
10part_2	10	81.7384	3,150	0.005
10part_3	10	136.74	4,005	0.006
10parts_4	10	75.7821	4,082	0.007
11part	11	110.607	19,264	0.032
12part	12	139.084	53,626	0.115
13part	13	168.723	201,741	0.409
14part	14	156.522	388,614	0.747
15part	15	139.061	577,447	1.092
15part_2-S	15	118.238	834,240	1.577

problem-specific exact method is required—which motivates the branch-and-bound algorithm developed in the remainder of this section. (As a side check, the MILP mix-batch optima coincide with those of our branch-and-bound, e.g. 59.47 on `13part` with $\bar{M} = 2$, confirming both implementations solve the same problem.)

B. Single-Machine Oracle

Table II reports the oracle on the benchmark instances. For every instance the oracle terminates with an empty active list and therefore returns the proven optimum (optimality gap 0%). Instances with up to 15 parts are solved in well under two seconds, and synthetic instances with 16 and 17 parts (generated with $TF = RDD = 0.6$ on a 20×20 platform) are proven optimal within roughly 12 and 16 seconds, respectively. For $n \geq 18$ the oracle still returns a strong feasible solution within a short time limit, but the proof of optimality is not completed: the bottleneck is the lower bound, whose root value reaches only about 14% of the optimum for these instances.

C. Effect of the Diving Rule

The diving phase is intended both to obtain incumbents quickly and, when the active list grows beyond $N_{\max}$, to bound memory. We compare the depth-first dive (deepest node first, ties to the most balanced) against the balance-first rule (most balanced node first), reporting generated nodes and the peak size of the active list. Both rules leave the proven optimum unchanged. Table III shows that the depth-first dive is uniformly preferable: it reduces generated nodes by up to two orders of magnitude and shrinks the peak active list by three to four orders of magnitude (e.g. from $14{,}905$ to 23 on `15part` with $\bar{M} = 3$). The balance-first rule does not descend; it sweeps the frontier by load level, so the active list grows quickly and, on the larger instances, it fails to prove optimality within the time limit while the depth-first dive proves it in seconds. This confirms that a genuine depth-first descent, not a balance-first sweep, is what makes the diving mode effective for both incumbents and memory.

TABLE III

DIVING RULE: GENERATED NODES AND PEAK ACTIVE-LIST SIZE. BOTH RULES RETURN THE SAME OPTIMUM; † MARKS RUNS THAT DID NOT PROVE OPTIMALITY WITHIN A 30–35 S LIMIT (THE DEPTH-FIRST DIVE PROVES ALL FIVE).

Instance	$\bar{M}$	Balance-first		Depth-first	
		Nodes	Peak	Nodes	Peak
13part	3	97,990	30,469	5,119	366
13part	4	25,277	6,926	1,293	23
15part	3	93,975	14,905	3,399	23
16part	3	184,275†	122,602	52,173	4,413
18part	3	65,022†	129,940	29,255	33

TABLE IV

LOCAL-SEARCH REFINEMENT UNDER THE DEPTH-FIRST DIVE (GENERATED NODES). BOTH CONFIGURATIONS RETURN THE SAME OPTIMUM.

Instance	$\bar{M}$	Generated nodes	
		Default	No LS
13part	2	5,065	5,215
13part	3	5,119	6,127
13part	4	1,293	4,457
14part	2	6,397	5,877
14part	3	6,399	2,445
15part	2	9,865	9,881
15part	3	3,399	8,910
15part	4	0	1,546

D. Effect of the Local-Search Upper Bound

Under the depth-first dive, we next isolate the additional contribution of the local-search refinement (Section II-E.2) by toggling it while keeping everything else fixed. In every case both configurations return the same proven optimum. Table IV reports the generated nodes for $\bar{M} \in \{2, 3, 4\}$. Because the depth-first dive already reaches a balanced, near-optimal incumbent quickly, the marginal effect of the local search is now small and instance dependent: it still helps on some configurations (e.g. `15part` with $\bar{M} \in \{3, 4\}$) but is neutral or slightly adverse on others (e.g. `14part` with $\bar{M} = 3$). In other words, once the diving rule is corrected, the dive itself performs most of the incumbent tightening that the local search previously supplied, so the two are partly redundant. We keep the local search enabled by default because it is inexpensive and never increases the proven optimum.

E. A Negative Result on Free-Part Bounds

The node lower bound charges each free (unassigned) part a per-part tardiness floor, corresponding to processing it first and alone on an empty machine. A natural attempt to strengthen this term is a parallel positional bound: among the first k globally completed free parts, some machine must hold at least $\lceil k/\bar{M} \rceil$ of them, yielding a positional completion threshold. We implemented this bound and verified that it never changes the proven optimum. However, it produces *no* additional pruning: across the tested instances its value

TABLE V

HEAVY-MACHINE ORACLE BOUND ON LOOSE-DUE-DATE INSTANCES ($\bar{M} = 3$, 30 S LIMIT). "BIG CALLS" COUNTS EXACT ORACLE EVALUATIONS ON SUBSETS OF ≥ 12 PARTS. ‡ THE DISABLED-BOUND INCUMBENT IS SUBOPTIMAL.

Instance	$\bar{M}$	incumbent		big calls	
		off	on	off	on
20part_4-S	3	8.29‡	**4.63**	513	22
19part	3	27.56	**24.91**	102	0

TABLE VI

SCALING OF THE SINGLE-MACHINE ORACLE. THE GAP IS (OBJ − LB)/OBJ; $n \leq 17$ ARE PROVEN OPTIMAL.

Instance	n	Obj	LB	Gap	Time (s)
16part	16	304.217	304.217	0.0%	12.4
17part	17	206.31	206.31	0.0%	15.8
18part	18	232.734	133.637	42.6%	35
19part	19	352.096	124.476	64.7%	35
20part_3-S	20	224.652	62.18	72.3%	32
20part_4-S	20	240.99	67.98	71.8%	32

is dominated, position by position, by the per-part floor. The reason is structural. The per-part floor is independent of $\bar{M}$, whereas the positional threshold scales the completion time down by $\lceil k/\bar{M} \rceil$, since distributing parts over more machines makes them finish earlier. Spreading work across machines therefore weakens the positional bound exactly as it would strengthen a single-machine one. We also tested a marginal-cost bound based on oracle increments; it requires the oracle value Φ to be supermodular in the part set, which fails empirically. Consequently, cheap valid strengthenings of the free-part bound appear exhausted, and the default configuration disables the positional free-part bound.

F. Pruning Heavily Unbalanced Assignments

In contrast to the free-part bound, a bound on the *assigned* side does pay off. As described in Section II-D.4, the analytical machine bound collapses to nearly zero on a heavily loaded machine, so the assignment search is not deterred from building extremely unbalanced complete assignments—one machine holding most of the parts—even though such assignments are essentially never optimal. Evaluating their leaves requires an exact oracle call on a large part set, which is costly. The heavy-machine oracle bound prunes these subtrees before any such call.

The effect is pronounced on instances with loose due dates, where the overall tardiness is small and the analytical bounds are weakest. Table V reports two 20- and 19-part instances under $\bar{M} = 3$. With the heavy-machine bound disabled, the search spends essentially its entire time budget evaluating heavily unbalanced leaves (hundreds of exact oracle calls on subsets of twelve or more parts) and never escapes a poor incumbent. Enabling the bound collapses these large-subset calls (from 513 to 22, and from 102 to 0) and frees the budget to reach far better solutions: on `20part_4-S` the incumbent improves from 8.29 to 4.63. Notably, the value 8.29 reached without the bound is itself *suboptimal*—the search had simply never reached the better region. On instances that already prove optimality within the limit ($n \leq 18$), the heavy machine threshold is rarely crossed and the overhead is negligible (e.g. `18part`, $\bar{M} = 3$, 1.04 s versus 1.23 s, same proven optimum).

This is a useful corrective to the common assumption that such large instances are purely lower-bound-limited: a meaningful part of the difficulty was the search exhausting its budget on provably irrelevant unbalanced leaves, which a cheap oracle-based bound on heavily loaded machines removes.

G. Scaling Behavior

Table VI summarizes the single-machine oracle on larger instances under a short time limit used only to illustrate the size at which the proof of optimality stops completing; longer limits prove more. Instances up to $n = 17$ are proven optimal in seconds. For $n = 18$ to 20 the oracle returns a strong incumbent but a large optimality gap remains within the short limit, again because the bottleneck is the lower bound rather than the incumbent. Closing this gap for $n \geq 18$ would require a fundamentally stronger bounding scheme, such as branch-and-price over true batch columns, which is left for future work.

V. CONCLUSIONS

We presented an exact branch-and-bound algorithm for the identical parallel-machine one-dimensional AM batch scheduling problem with the total tardiness objective. The algorithm branches on part-to-machine assignments and evaluates each complete assignment by decomposing it into independent single-machine subproblems, each solved exactly by a dedicated oracle equipped with valid parallel and positional lower bounds and dominance pruning. At the parallel-machine level, node bounds combine oracle-based bounds on assigned parts with per-part floors on free parts, and a memoization pool both eliminates redundant oracle calls and strengthens node bounds.

The computational study yields three findings. First, the single-machine oracle proves optimality for up to 17 parts within seconds, with the lower bound, not the incumbent, limiting larger sizes. Second, the diving rule used inside the search must be a genuine depth-first descent: replacing a balance-first selection, which sweeps the frontier by load level, with a deepest-node rule (ties broken toward balance) reduces both the search-tree size and the peak active-list size by up to several orders of magnitude and lets the algorithm prove optimality on instances the balance-first rule cannot, all without affecting the proven optimum. Once this rule is corrected, the move-based local-search upper bound becomes a minor, inexpensive refinement that the dive largely subsumes. Third, a parallel positional free-part bound, although valid, is structurally dominated by the per-part floor and gives no additional pruning, indicating that cheap free-part bound strengthenings are largely exhausted. Fourth, a

complementary bound on the *assigned* side—a short, budget-limited oracle run on any heavily loaded machine—prunes the extremely unbalanced assignments that the analytical bound fails to detect, eliminating wasted large-subset oracle calls and substantially improving incumbents on loose-due-date instances (e.g. from a suboptimal 8.29 to 4.63 on a 20-part instance); this reveals that part of the apparent gap stemmed from search effort wasted on provably irrelevant unbalanced leaves rather than from the bound alone. Closing the remaining gap for $n \geq 18$ nonetheless still calls for a fundamentally stronger bounding scheme, such as branch-and-price over true batch columns, which we leave for future work.

REFERENCES

[1] M. Pinto, C. Silva, M. Thürer, and S. Moniz, "Nesting and scheduling optimization of additive manufacturing systems: Mapping the territory," *Computers & Operations Research*, vol. 165, p. 106592, May 2024.

[2] H. Wu, C. Yu, and S. Yu, "Nesting and scheduling for parallel additive manufacturing machines with uncertain processing times: A simulation-optimisation approach," *International Journal of Production Research*, pp. 1–30, Jun. 2025.

[3] Z. Mao, E. Fu, D. Huang, K. Fang, and L. Chen, "Combinatorial Benders decomposition for single machine scheduling in additive manufacturing with two-dimensional packing constraints," *European Journal of Operational Research*, vol. 317, no. 3, pp. 890–905, Sep. 2024.

[4] B. Zipfel, F. Tamke, and L. Kuttner, "A new branch-and-cut approach for integrated planning in additive manufacturing," *European Journal of Operational Research*, p. S0377221724008439, Nov. 2024.

[5] P. J. Nascimento, C. Silva, C. H. Antunes, and S. Moniz, "Optimal decomposition approach for solving large nesting and scheduling problems of additive manufacturing systems," *European Journal of Operational Research*, vol. 317, no. 1, pp. 92–110, Aug. 2024.

[6] P. J. Nascimento, C. Silva, C. H. Antunes, C. T. Maravelias, and S. Moniz, "Improving the efficiency of Logic-based Benders Decomposition for p-batch scheduling problems with two-dimensional packing," *European Journal of Operational Research*, p. S0377221726000767, Jan. 2026.

[7] Y. Che, K. Hu, Z. Zhang, and A. Lim, "Machine scheduling with orientation selection and two-dimensional packing for additive manufacturing," *Computers & Operations Research*, vol. 130, p. 105245, Jun. 2021.

[8] S. K. Tangudu and M. E. Kurz, "A branch and bound algorithm to minimise total weighted tardiness on a single batch processing machine with ready times and incompatible job families," *Production Planning & Control*, vol. 17, no. 7, pp. 728–741, Oct. 2006.

[9] E. L. Lawler, "A "Pseudopolynomial" Algorithm for Sequencing Jobs to Minimize Total Tardiness," in *Annals of Discrete Mathematics*. Elsevier, 1977, vol. 1, pp. 331–342.

[10] C. Yu, A. Matta, Q. Semeraro, and J. Lin, "Mathematical Models for Minimizing Total Tardiness on Parallel Additive Manufacturing Machines," *IFAC-PapersOnLine*, vol. 55, no. 10, pp. 1521–1526, 2022.

APPENDIX

This appendix derives the positional lower bound used in Section II-D.3. Consider an arbitrary part set $\mathcal{Q} \subseteq \mathcal{J}$ to be processed on a single machine starting from time zero. Let $q = |\mathcal{Q}|$. For each part $j \in \mathcal{Q}$, let

$$a_j = l_j w_j$$

denote its projected area. Sort the parts in $\mathcal{Q}$ by projected area, height, and volume:

$$a_{(1)} \leq a_{(2)} \leq \cdots \leq a_{(q)},$$
$$h_{[1]} \leq h_{[2]} \leq \cdots \leq h_{[q]},$$
$$v_{\langle 1 \rangle} \leq v_{\langle 2 \rangle} \leq \cdots \leq v_{\langle q \rangle}.$$

For each position $k = 1, \ldots, q$, define

$$A_k = \sum_{r=1}^{k} a_{(r)}, \qquad \beta_k = \left\lceil \frac{A_k}{LW} \right\rceil,$$

and

$$\underline{V}_k = \sum_{r=1}^{k} v_{\langle r \rangle}.$$

The quantity A_k is the minimum possible total projected area of any k parts in $\mathcal{Q}$. Therefore, before the k-th part can be completed, at least β_k batches must have been processed. This gives a setup-time lower bound of $\beta_k S$. Similarly, the total processed volume before the k-th completion must be at least the sum of the k smallest volumes, namely $\underline{V}_k$. Hence, the volume-dependent processing-time contribution is at least $V\underline{V}_k$.

It remains to lower bound the cumulative height-dependent contribution. Consider the first k completed parts in any feasible schedule. These parts must be distributed over at least β_k nonempty batches before the k-th completion. Let H_b denote the maximum height of batch b. The cumulative height contribution of these batches is therefore at least the optimal value of the following auxiliary problem:

$$H_k^\star = \min \sum_{b=1}^{r} H_b$$

subject to

$$B_b \subseteq \{1, \ldots, k\}, \qquad b = 1, \ldots, r,$$
$$\bigcup_{b=1}^{r} B_b = \{1, \ldots, k\},$$
$$B_b \cap B_{b'} = \emptyset, \qquad 1 \leq b < b' \leq r,$$
$$B_b \neq \emptyset, \qquad b = 1, \ldots, r,$$
$$\sum_{j \in B_b} a_j \leq LW, \qquad b = 1, \ldots, r,$$
$$H_b = \max_{j \in B_b} h_j, \qquad b = 1, \ldots, r,$$
$$r \geq \beta_k.$$

This auxiliary problem is still combinatorial. We therefore relax it by fixing the number of groups to β_k and removing the capacity constraints. The relaxed problem is

$$\widetilde{H}_k = \min \sum_{b=1}^{\beta_k} H_b$$

subject to

$$G_b \subseteq \{1, \ldots, k\}, \qquad b = 1, \ldots, \beta_k,$$
$$\bigcup_{b=1}^{\beta_k} G_b = \{1, \ldots, k\},$$
$$G_b \cap G_{b'} = \emptyset, \qquad 1 \leq b < b' \leq \beta_k,$$

$$G_b \neq \emptyset, \qquad b = 1, \ldots, \beta_k,$$

$$H_b = \max_{j \in G_b} h_{[j]}, \qquad b = 1, \ldots, \beta_k.$$

By construction, this is a relaxation of the auxiliary height problem. Therefore,

$$\widetilde{H}_k \leq H_k^\star.$$

Lemma 1.: For each $k = 1, \ldots, q$, the optimal value of the relaxed height problem is

$$\widetilde{H}_k = \sum_{r=1}^{\beta_k - 1} h_{[r]} + h_{[k]}.$$

Proof.: Since the β_k groups must be nonempty, each group contributes at least the height of one assigned part through its group maximum. To minimize the sum of group maxima, $\beta_k - 1$ groups should be made singleton groups containing the $\beta_k - 1$ smallest heights $h_{[1]}, \ldots, h_{[\beta_k - 1]}$. All remaining parts are assigned to the last group, whose maximum height is $h_{[k]}$. Therefore,

$$\widetilde{H}_k = \sum_{r=1}^{\beta_k - 1} h_{[r]} + h_{[k]}.$$

This proves the lemma. □

Define

$$\underline{H}_k = \widetilde{H}_k = \sum_{r=1}^{\beta_k - 1} h_{[r]} + h_{[k]}.$$

Combining the setup, volume, and height contributions, define

$$\underline{C}_k = \beta_k S + V \underline{V}_k + U \underline{H}_k, \qquad k = 1, \ldots, q.$$

Proposition 2.: Let $C_{(k)}^{\text{real}}$ denote the completion time of the k-th completed part among the parts in $\mathcal{Q}$ in any feasible single-machine schedule. Then,

$$C_{(k)}^{\text{real}} \geq \underline{C}_k, \qquad k = 1, \ldots, q.$$

Proof.: Fix any $k \in \{1, \ldots, q\}$ and consider the first k completed parts in an arbitrary feasible schedule.

First, these k parts have total projected area at least

$$A_k = \sum_{r=1}^{k} a_{(r)}.$$

Since each batch has capacity LW, at least

$$\beta_k = \left\lceil \frac{A_k}{LW} \right\rceil$$

batches must have been processed before the k-th completion. Hence, the cumulative setup contribution is at least $\beta_k S$.

Second, the total processed volume before the k-th completion must be at least the sum of the k smallest volumes in $\mathcal{Q}$:

$$\underline{V}_k = \sum_{r=1}^{k} v_{\langle r \rangle}.$$

Thus, the cumulative volume-dependent contribution is at least $V \underline{V}_k$.

Third, the cumulative height-dependent contribution before the k-th completion is at least $U H_k^\star$. Since

$$H_k^\star \geq \widetilde{H}_k = \underline{H}_k,$$

this contribution is further bounded below by $U \underline{H}_k$.

Combining the three components gives

$$C_{(k)}^{\text{real}} \geq \beta_k S + V \underline{V}_k + U \underline{H}_k = \underline{C}_k.$$

This proves the proposition. □

To transform the completion-time lower bounds into a total tardiness lower bound, sort the due dates of the parts in $\mathcal{Q}$ in nondecreasing order:

$$d_{[1]}^{\uparrow} \leq d_{[2]}^{\uparrow} \leq \cdots \leq d_{[q]}^{\uparrow}.$$

Since the tardiness function is nondecreasing in completion time, the minimum tardiness implied by the positional completion-time lower bounds is obtained by matching smaller completion-time lower bounds with smaller due dates. Hence,

$$LB^{\text{pos}}(\mathcal{Q}) = \sum_{k=1}^{q} \max\left\{0,\ \underline{C}_k - d_{[k]}^{\uparrow}\right\}$$

is a valid lower bound on the optimal single-machine total tardiness $\Phi(\mathcal{Q})$.

Applying this expression to $\mathcal{Q} = \mathcal{Q}_m(N) = \mathcal{S}_m(N)$ yields the machine-wise positional lower bound $LB_m^{\text{pos}}(\mathcal{Q}_m(N))$ used in the main algorithm.

APPENDIX

This appendix describes the exact single-machine scheduling oracle used in the parallel-machine branch-and-bound algorithm. For a given part subset $\mathcal{Q} \subseteq \mathcal{J}$, the oracle solves the single-machine one-dimensional AM batch scheduling problem with total tardiness minimization. The optimal objective value returned by the oracle is denoted by $\Phi(\mathcal{Q})$.

A. Single-Machine Subproblem

Given a part subset $\mathcal{Q}$, the single-machine subproblem is to partition the parts in $\mathcal{Q}$ into feasible batches and sequence these batches on one AM machine. A batch $B \subseteq \mathcal{Q}$ is feasible if

$$\sum_{j \in B} l_j w_j \leq LW. \tag{18}$$

The processing time of a feasible batch B is

$$p(B) = S + V \sum_{j \in B} v_j + U \max_{j \in B} h_j. \tag{19}$$

If batch B is completed at time $C(B)$, then every part $j \in B$ has completion time $C(B)$ and tardiness

$$T_j = \max\{0, C(B) - d_j\}. \tag{20}$$

The oracle returns

$$\Phi(\mathcal{Q}) = \min \sum_{j \in \mathcal{Q}} T_j. \tag{21}$$

If $\mathcal{Q} = \emptyset$, we define $\Phi(\mathcal{Q}) = 0$.

B. Node Representation

The oracle is implemented by a branch-and-bound algorithm that builds the batch sequence from time zero. Each node N is represented by

$$N = \big(\mathcal{B}(N), \mathcal{U}(N), C_N, TT_N\big), \tag{22}$$

where $\mathcal{B}(N)$ is the sequence of batches already fixed, $\mathcal{U}(N)$ is the set of unscheduled parts, C_N is the completion time of the last fixed batch, and TT_N is the total tardiness contributed by the already scheduled parts.

The root node is

$$N^0 = (\emptyset, \mathcal{Q}, 0, 0). \tag{23}$$

A node is terminal if

$$\mathcal{U}(N) = \emptyset. \tag{24}$$

At a terminal node, TT_N is the total tardiness of a complete feasible single-machine schedule.

C. Branching Rule

At a nonterminal node N, child nodes are generated by selecting a nonempty feasible subset $B \subseteq \mathcal{U}(N)$ as the next batch. The subset must satisfy

$$\sum_{j \in B} l_j w_j \le LW. \tag{25}$$

For each feasible subset B, a child node N' is generated as follows:

$$\mathcal{B}(N') = \mathcal{B}(N) \oplus B, \tag{26}$$

where $\oplus$ denotes appending B to the current batch sequence,

$$\mathcal{U}(N') = \mathcal{U}(N) \setminus B, \tag{27}$$

$$C_{N'} = C_N + p(B), \tag{28}$$

and

$$TT_{N'} = TT_N + \sum_{j \in B} \max\{0, C_{N'} - d_j\}. \tag{29}$$

This branching scheme is complete because any feasible single-machine batch schedule can be uniquely represented as a sequence of feasible batches selected from the remaining parts at each stage.

D. Initial Upper Bound

Before the branch-and-bound search starts, an initial feasible solution is constructed by a fast heuristic. The parts in $\mathcal{Q}$ are first sorted in nondecreasing order of due dates. The heuristic then scans the sorted list and assigns each part to the current batch if the platform capacity constraint remains satisfied. Otherwise, the current batch is closed and a new batch is opened. The resulting batch sequence is evaluated exactly by (19), and its total tardiness is used as the initial upper bound UB.

In the implementation, several simple constructive rules can be used, such as earliest due date, largest projected area, largest volume, and largest height. The best schedule obtained from these rules is selected as the initial incumbent.

E. Parallel Lower Bound

At a node N, the parallel lower bound relaxes the single-machine restriction by assuming that each unscheduled part can be processed independently as a singleton batch immediately after the current completion time C_N. For each part $j \in \mathcal{U}(N)$, the earliest completion time under this relaxation is

$$C_N + S + Vv_j + Uh_j. \tag{30}$$

Therefore, the parallel lower bound is defined as

$$LB_{\mathrm{par}}(N) = TT_N + \sum_{j \in \mathcal{U}(N)} \max\{0, C_N + S + Vv_j + Uh_j - d_j\}. \tag{31}$$

This bound is valid because no feasible continuation can complete part j before processing at least its own volume and height contribution together with one setup operation after time C_N.

F. Positional Lower Bound

Although $LB_{\mathrm{par}}(N)$ is inexpensive, it ignores the sequential nature of the remaining decisions, since it lets every unscheduled part complete as its own singleton batch immediately after C_N. A stronger bound is obtained by applying the positional argument of Appendix to the unscheduled part set $\mathcal{U}(N)$, shifted by the current completion time C_N.

Let $\underline{C}_1 \le \underline{C}_2 \le \cdots \le \underline{C}_{|\mathcal{U}(N)|}$ be the positional completion-time lower bounds of Appendix computed over the part set $\mathcal{U}(N)$, and let $d^{\uparrow}_{[1]} \le \cdots \le d^{\uparrow}_{[|\mathcal{U}(N)|]}$ be the nondecreasing due dates of $\mathcal{U}(N)$. Because every unscheduled part is processed after C_N, the completion time of the k-th completed unscheduled part is at least $C_N + \underline{C}_k$. Matching the shifted completion-time bounds with the sorted due dates yields the positional lower bound

$$LB_{\mathrm{pos}}(N) = TT_N + \sum_{k=1}^{|\mathcal{U}(N)|} \max\big\{0,\ C_N + \underline{C}_k - d^{\uparrow}_{[k]}\big\}. \tag{32}$$

Its validity follows from Proposition 2 of Appendix , applied to $\mathcal{U}(N)$, together with the fact that adding the common offset C_N to every completion-time bound preserves the inequality.

G. Combined Lower Bound

At each node, both bounds are computed and the larger one is used:

$$LB(N) = \max\big\{LB_{\mathrm{par}}(N),\ LB_{\mathrm{pos}}(N)\big\}. \tag{33}$$

Since $LB_{\mathrm{par}}(N)$ and $LB_{\mathrm{pos}}(N)$ are both valid lower bounds on the total tardiness of any complete schedule extending N, their maximum is also valid.

H. Dominance Rule

A dominance rule is used to remove redundant partial schedules. Let u and v be two nodes. Let $\mathcal{S}_u$ and $\mathcal{S}_v$ denote the sets of already scheduled parts at these two nodes. Node u dominates node v, denoted by $u \prec v$, if

$$\mathcal{S}_u = \mathcal{S}_v, \tag{34}$$

$$C_u \le C_v, \tag{35}$$

and

$$TT_u \le TT_v, \tag{36}$$

with at least one inequality being strict.

Proposition 1. *If node u dominates node v, then node v can be safely pruned.*

Proof. Since $\mathcal{S}_u = \mathcal{S}_v$, the two nodes have the same unscheduled part set and admit the same feasible continuations. Consider any feasible continuation of node v. Applying the same continuation to node u is also feasible. The appended batches have identical processing times in both continuations. Since $C_u \le C_v$, every future batch completion time from node u is no later than the corresponding completion time from node v. Thus, the future tardiness generated from node u is no larger than that generated from node v. Together with $TT_u \le TT_v$, the total tardiness of the complete schedule obtained from node u is no larger than that obtained from node v. Therefore, node v cannot lead to a better solution than node u and can be safely pruned. □

In the implementation, nondominated labels are maintained for each scheduled part set. A newly generated node is discarded if it is dominated by an existing label with the same scheduled set. Otherwise, it is inserted into the active node list, and any existing labels dominated by the new node are removed.

I. Node Selection

The active nodes are stored in a list $\mathcal{L}$. The oracle supports three node selection strategies.

In depth-first search, the selected node is one of the deepest active nodes:

$$N^* = \arg\max_{N \in \mathcal{L}} |\mathcal{B}(N)|. \tag{37}$$

In best-first search, the selected node has the smallest lower bound:

$$N^* = \arg\min_{N \in \mathcal{L}} LB(N). \tag{38}$$

The adaptive strategy switches between depth-first and best-first search. It uses best-first search by default. If the number of active nodes exceeds a threshold $N_{\max}$, it switches to depth-first search to reduce memory usage. Once the number of active nodes decreases below $N_{\min}$, it switches back to best-first search.

J. Overall Procedure

The exact oracle proceeds as follows.

1) Construct an initial feasible batch sequence and set its total tardiness as the initial upper bound UB.
2) Initialize the root node
$$N^0 = (\emptyset, \mathcal{Q}, 0, 0)$$
and insert it into the active node list $\mathcal{L}$.
3) While $\mathcal{L}$ is not empty, select a node N according to the node selection strategy.
4) Compute $LB(N)$. If
$$LB(N) \ge UB,$$
prune node N.
5) If $\mathcal{U}(N) = \emptyset$, node N is terminal. If
$$TT_N < UB,$$
update the incumbent schedule and set $UB \leftarrow TT_N$. Then discard node N.
6) If $\mathcal{U}(N) \neq \emptyset$, enumerate all nonempty feasible subsets $B \subseteq \mathcal{U}(N)$ satisfying
$$\sum_{j \in B} l_j w_j \le LW.$$
For each feasible subset, generate the corresponding child node.
7) Apply the dominance rule to each generated child node. If the child node is dominated, discard it. Otherwise, insert it into $\mathcal{L}$ and update the dominance labels.
8) When $\mathcal{L}$ becomes empty, return the incumbent schedule and its objective value UB.

K. Optimality Guarantee

The oracle enumerates all feasible batch sequences through the branching scheme. A node is pruned only when its lower bound is no smaller than the current incumbent upper bound or when it is dominated by another node that has scheduled the same part set with no larger completion time and no larger accumulated tardiness. Both pruning mechanisms are valid. Therefore, when the active node list becomes empty, every feasible schedule has either been evaluated explicitly or proven unable to improve the incumbent. The returned incumbent schedule is thus globally optimal, and its objective value is $\Phi(\mathcal{Q})$.

If the oracle is terminated by a time limit before the active node list becomes empty, the returned incumbent remains feasible, but global optimality is not guaranteed.